

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Программирование графических процессоров»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 2

«Особенности написания программ на CUDA»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является изучение особенностей разработки программ с использованием CUDA, освоение принципов управления памятью GPU, взаимодействия центрального процессора и графического устройства, обработки ошибок CUDA, синхронизации потоков и измерения времени выполнения вычислений на GPU.

1.2. Задачи

- Изучить способы выделения, копирования и освобождения памяти на GPU.
- Реализовать передачу данных между памятью CPU и GPU.
- Исследовать зависимость скорости копирования данных от объёма передаваемого одномерного массива.
- Реализовать CUDA-ядро для сложения двух векторов.
- Измерить время выполнения сложения двух векторов средствами языка C/C++ и средствами CUDA Events.
- Сравнить результаты измерений и объяснить различия между ними.
- Освоить базовые средства обработки ошибок CUDA.
- Изучить способы синхронизации CPU, GPU и потоков внутри CUDA-блока.
- Рассмотреть типичную структуру CUDA-программы.
- Ответить на контрольные вопросы.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Архитектура взаимодействия CPU и GPU

CUDA-программа выполняется во взаимодействии двух вычислительных устройств: центрального процессора CPU, называемого *host*, и графического процессора GPU, называемого *device*. Управляющая часть программы выполняется на CPU, а ресурсоёмкие параллельные вычисления могут передаваться на GPU в виде CUDA-ядер. Для выполнения вычислений данные необходимо передать из памяти CPU в память GPU, после чего GPU выполняет обработку, а результаты при необходимости копируются обратно на CPU.

2.2. Управление памятью GPU

Для динамического выделения памяти на устройстве используется функция `cudaMalloc()`, а для освобождения ранее выделенной памяти — `cudaFree()`. Передача данных между *host* и *device* выполняется функцией `cudaMemcpy()`. В качестве направления копирования могут использоваться `cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`, `cudaMemcpyDeviceToDevice` и `cudaMemcpyHostToHost`. Таким образом, типичный обмен данными состоит из выделения памяти на GPU, копирования входных данных с CPU на GPU, выполнения вычислений, копирования результата обратно на CPU и освобождения памяти.

2.3. CUDA-ядро и организация потоков

CUDA-ядро объявляется со спецификатором `__global__`. Такая функция вызывается с CPU, но выполняется на GPU множеством потоков параллельно. Каждый поток получает собственный идентификатор, который может быть вычислен с использованием встроенных переменных `threadIdx`, `blockIdx` и `blockDim`. Глобальный индекс потока обычно определяется выражением $i = threadIdx.x + blockIdx.x * blockDim.x$. Это позволяет сопоставить отдельный поток с конкретным элементом обрабатываемого массива.

2.4. Сложение двух векторов на GPU

Сложение двух векторов является типичным примером параллельной обработки. Для каждого индекса i вычисляется значение $c[i] = a[i] + b[i]$. Каждый элемент результата может обрабатываться отдельным CUDA-потокom независимо от других элементов. Функция-обёртка выделяет память на GPU, копирует исходные массивы, запускает ядро, получает результат и освобождает память.

2.5. Обработка ошибок CUDA

Большинство функций CUDA возвращает значение типа `cudaError_t`. При успешном выполнении возвращается `cudaSuccess`. Для получения кода последней ошибки используется `cudaGetLastError()`, а для получения текстового описания ошибки — `cudaGetErrorString()`. Проверка результата выполнения CUDA-функций позволяет обнаруживать ошибки выделения памяти, передачи данных, некорректных указателей, неправильных параметров запуска ядра и другие проблемы.

2.6. Измерение времени выполнения

Для измерения времени выполнения операций непосредственно на GPU используются CUDA Events. Объект `cudaEvent_t` позволяет установить временные точки до и после исследуемого участка программы. После синхронизации с событием `stop` функция `cudaEventElapsedTime()` возвращает продолжительность выполнения в миллисекундах. Такой способ измерения предпочтительнее обычного измерения времени на CPU при исследовании отдельных операций GPU, поскольку учитывает специфику асинхронного характера выполнения CUDA.

2.7. Синхронизация в CUDA

Для синхронизации CPU с завершением операций GPU используется `cudaDeviceSynchronize()`. Внутри CUDA-ядра для синхронизации потоков одного блока используется `__syncthreads()`. Данная функция создаёт барьер, который поток не может пройти, пока остальные потоки того же блока не достигнут этого места. Дополнительно существуют функции `__threadfence_block()`, `__threadfence()` и `__threadfence_system()`, предназначенные для обеспечения видимости операций с памятью на разных уровнях.

2.8. Типичная структура CUDA-программы

Типичная CUDA-программа включает объявление переменных и прототипов ядер, функцию `main()`, выделение памяти на GPU с помощью `cudaMalloc()`, передачу данных с CPU на GPU посредством `cudaMemcpy()`, определение конфигурации запуска ядра, вызов `kernel<<>>()`, получение результатов на CPU и освобождение ресурсов. Само ядро содержит параллельный алгоритм обработки данных и при необходимости операции синхронизации потоков.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Подготовка программы

Для выполнения практической работы используется программа на C++ с расширением `.cu`, компилируемая компилятором NVIDIA CUDA Compiler (`nvcc`). Программа содержит функции проверки ошибок CUDA, измерения времени копирования данных и CUDA-ядро сложения двух векторов.

3.2. Исследование передачи данных между CPU и GPU

- Для эксперимента формируются одномерные массивы типа `float` различного размера.
- Память массива выделяется на CPU, а соответствующая память для GPU выделяется с помощью `cudaMalloc()`.
- Исходный массив копируется с CPU на GPU посредством `cudaMemcpy()` с направлением `cudaMemcpyHostToDevice`.
- Для обратного эксперимента результат копируется с GPU на CPU с направлением `cudaMemcpyDeviceToHost`.
- Для каждого объёма данных измеряется время передачи.
- На основании измерений рассчитывается скорость передачи по формуле $V = S / t$, где S — объём переданных данных, а t — время передачи.

3.3. Реализация сложения двух векторов

- Создаются три массива float: два входных массива a и b и массив результата c.
- Память для массивов выделяется на CPU и GPU.
- Массивы a и b копируются в память GPU.
- Запускается CUDA-ядро sumKernel().
- Каждый поток вычисляет глобальный индекс элемента.
- Для соответствующего индекса выполняется операция $c[i] = a[i] + b[i]$.
- Результат копируется из памяти GPU обратно на CPU.
- Результат проверяется сравнением с ожидаемыми значениями.
- После завершения эксперимента память CPU и GPU освобождается.

3.4. Измерение времени с помощью C/C++

Для измерения времени выполнения используется стандартный механизм высокоточного системного таймера CPU. Время фиксируется непосредственно перед запуском исследуемого участка и сразу после его завершения. Полученная разница характеризует время, наблюдаемое CPU, и может включать дополнительные задержки, связанные с запуском CUDA-операций и синхронизацией.

3.5. Измерение времени с помощью CUDA Events

Для измерения времени непосредственно на GPU создаются два объекта `cudaEvent_t` — `start` и `stop`. Перед запуском исследуемого ядра вызывается `cudaEventRecord(start)`, после запуска фиксируется `stop`. Затем выполняется `cudaEventSynchronize(stop)`, после чего `cudaEventElapsedTime()` возвращает время между событиями в миллисекундах. Такой подход позволяет измерить непосредственно временной интервал выполнения GPU-операции.

3.6. Контроль ошибок

После основных операций CUDA выполняется проверка возвращаемого значения `cudaError_t`. При обнаружении ошибки её код преобразуется в текст с помощью `cudaGetErrorString()`. Для проверки ошибок запуска CUDA-ядра дополнительно используется `cudaGetLastError()`. Такой подход позволяет определить момент возникновения ошибки и получить диагностическое описание.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Программа исследования скорости копирования

Для исследования зависимости скорости передачи данных от объёма массива выполняется серия экспериментов с увеличением количества элементов. Для каждого размера создаётся массив `float`, производится передача `Host-to-Device` и `Device-to-Host`, после чего фиксируется время операции. Скорость вычисляется как отношение объёма переданных данных к времени передачи.

4.2. Результаты исследования копирования

В общем случае при увеличении объёма передаваемого массива наблюдается рост времени копирования, однако скорость передачи не возрастает линейно. Для небольших массивов существенную часть времени составляют накладные расходы на запуск операции передачи. При увеличении размера массива эти накладные расходы становятся менее значительными относительно общего объёма данных, поэтому скорость приближается к некоторому предельному значению, определяемому пропускной способностью интерфейса и особенностями используемой памяти.

4.3. Пример таблицы измерений скорости копирования

- Объём данных 1 МВ — измеряется время передачи CPU → GPU и GPU → CPU.
- Объём данных 4 МВ — измеряется время передачи CPU → GPU и GPU → CPU.
- Объём данных 16 МВ — измеряется время передачи CPU → GPU и GPU → CPU.
- Объём данных 64 МВ — измеряется время передачи CPU → GPU и GPU → CPU.
- Объём данных 256 МВ — измеряется время передачи CPU → GPU и GPU → CPU.
- Полученные значения должны использоваться для построения графика зависимости скорости передачи от объёма данных.
- Точные численные результаты зависят от модели GPU, типа системной памяти, интерфейса PCI Express и конфигурации конкретного компьютера.

4.4. Проверка сложения двух векторов

В ходе выполнения программы два массива передаются на GPU, после чего запускается CUDA-ядро сложения. Для каждого элемента вычисляется сумма соответствующих элементов двух входных массивов. После возврата результата на CPU выполняется проверка полученного массива. При корректной работе CUDA-ядра все элементы результата совпадают с эталонными значениями, вычисленными на CPU.

4.5. Сравнение измерения времени средствами CPU и CUDA Events

Измерение времени средствами CPU и CUDA Events может давать различные результаты. Причиной является асинхронная модель выполнения CUDA: вызов ядра с CPU может завершиться до фактического завершения вычислений на GPU. Поэтому простой замер времени вокруг запуска kernel без синхронизации не обязательно отражает реальное время выполнения GPU. CUDA Events устанавливаются непосредственно в потоке выполнения CUDA и позволяют измерить интервал между событиями на устройстве. Для корректного измерения с CPU требуется дополнительно вызвать `cudaDeviceSynchronize()` перед фиксацией конечного времени.

4.6. Анализ влияния размера массива

При обработке небольшого количества элементов использование GPU может не давать заметного выигрыша по сравнению с CPU из-за накладных расходов на выделение памяти, передачу данных и запуск ядра. При увеличении объёма данных параллельная архитектура GPU позволяет одновременно обрабатывать большое количество элементов, поэтому преимущества GPU становятся более выраженными. Однако если учитывать полный цикл обработки, включая копирование данных между CPU и GPU, итоговое ускорение может быть существенно меньше ускорения непосредственно вычислительного ядра.

4.7. Обработка ошибок в программе

В программе после операций `cudaMalloc()`, `cudaMemcpy()` и запуска ядра выполняется проверка результата. При успешном выполнении функция возвращает `cudaSuccess`. При возникновении проблемы выводится строковое описание ошибки с использованием `cudaGetErrorString()`. Такой механизм позволяет выявлять ошибки выделения памяти, неправильные указатели, некорректные параметры и ошибки выполнения CUDA-операций.

4.8. Контрольные вопросы

- **1. Перечислите виды синхронизации.** В CUDA используются синхронизация CPU с GPU с помощью `cudaDeviceSynchronize()`, барьерная синхронизация потоков одного блока с помощью `__syncthreads()`, а также механизмы обеспечения видимости операций с памятью `__threadfence_block()`, `__threadfence()` и `__threadfence_system()`.
- **2. Какие основные ошибки возникают при исполнении кода на CUDA?** К основным ошибкам относятся `cudaErrorMemoryAllocation` при невозможности выделить память, `cudaErrorInvalidValue` при передаче некорректного параметра, `cudaErrorInvalidDevicePointer` при использовании неправильного указателя устройства, `cudaErrorInvalidMemcpyDirection` при ошибочном направлении копирования, `cudaErrorInitializationError` при проблемах инициализации CUDA и ошибки, возникающие вследствие некорректного запуска или выполнения ядра.
- **3. Опишите типичную структуру CUDA-программы.** CUDA-программа обычно содержит функцию `main()`, выделение памяти на GPU, копирование входных данных с CPU на GPU, определение конфигурации запуска, вызов CUDA-ядра, синхронизацию при необходимости, копирование результатов обратно на CPU, проверку результатов и освобождение памяти. Само ядро содержит параллельный алгоритм, выполняемый потоками GPU.

V. ВЫВОДЫ

В результате выполнения практической работы были изучены основные особенности разработки программ на CUDA. Освоены операции выделения и освобождения памяти GPU с использованием `cudaMalloc()` и `cudaFree()`, а также передача данных между CPU и GPU посредством `cudaMemcpy()`. Реализовано параллельное сложение двух векторов с использованием CUDA-ядра, в котором каждый поток обрабатывает отдельный элемент массива. Были рассмотрены средства контроля ошибок `cudaError_t`, `cudaGetLastError()` и `cudaGetErrorString()`, позволяющие диагностировать проблемы при выполнении CUDA-программы. Также изучены механизмы синхронизации CPU и GPU, потоков внутри блока и операции обеспечения видимости данных в памяти. Для измерения времени выполнения GPU-операций рассмотрены CUDA Events. Эксперимент со скоростью передачи показал, что увеличение объёма данных приводит к росту времени копирования, однако относительное влияние накладных расходов уменьшается при работе с большими массивами. Сравнение способов измерения времени показало, что CUDA Events позволяют корректнее определить продолжительность выполнения GPU-операции, тогда как измерение средствами CPU требует учёта асинхронного характера CUDA. Таким образом, были закреплены основные принципы построения CUDA-программ: управление памятью, передача данных между host и device, запуск параллельных ядер, обработка ошибок, синхронизация и профилирование времени выполнения.